

day 3 CPU pipeline → Instruction-level Parallelism.

ISA - Contact between Software & the processor.

$C = a + b;$

LOAD R1, [a]

LOAD R2, [b]

ADD R3, R1, R2

STORE [c], R3

Program Counter (PC)

↳ keeps records of the location of the next instruction the CPU should fetch.

Assembly: human-readable form of machine instructions

Instruction Cycle:

a) If-Instruction Fetch - [get the instruction]

- CPU needs to know: Which instruction to execute next?

- Program Counter (PC) contains the address of the next instruction.

Fetch

↓
Decode

↓
Execute.

↓
Memory.

↓
Write Back

↓
Next Instruction

b) ID - Instruction Decode

understand the instructions.

c) EX - execute.

perform the operation.

ds MEM- memory Access

- access the memory when the instruction requires it.

es WB- Write Back.

- put back the result to the registers.

#CPU Pipeline:

Instead of completing ~~the~~ one instruction completely before starting the next, the CPU works on different stages of several instructions at the same time.

- latency: How ~~much~~ long does one instruction take from start to finish?
- time for one instruction.
- Throughput: How frequently can we finish instruction?
- amount of work completed per unit time.
- Pipeline defines the overlapping of stages for instruction execution.
- Pipeline is done to improve execution speed of programs.

8085: F1 E1 E2 E2 F3 E3

8086: F1 E1 E2 F4.
F2 F3

- Hazards in Pipelining:

- a) Data Hazard

- b) Control Hazard

- c) Structural Hazard